



SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE CODE: CSA1407

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

SCENARIO BASED TASK

Code of Conduct:

*I Sabitha. K(192524308) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
K.Sabitha

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and

SHRATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 - ANALYTICAL PROBLEM SOLVING

COMPLEXITY OF PROBLEM SOLVING

1) Scenario 1: Parsing Table Conflict in LL(1)

In a LL parser, the table is constructed using the first and follow set of the grammar. The first set determines the terminals that can appear at the beginning of strings derived from a production, while the follow set contains terminals that can appear immediately after non-terminals. During parsing table construction, each production is placed in the table based on set.

A conflict occurs when more than one production is entered into the same table cell for a non-terminal and an input symbol. This usually happens because the grammar contains left associations, common prefixes (left factoring problems), or is ambiguous. As a result, the parser cannot uniquely determine which product should be applied.

The parsing is deterministic, every table entry must contain only one production. Multiple entries make the parser non-deterministic.

The grammar should be modified by eliminating left recursion, performing left factoring, and ensuring that the first and follow set do not overlap. This converts the grammar into a valid LL grammar with a conflict-free parsing table.

2) Scenario : Incorrect Parse Tree Generation.

The syntax analyzer constructs a parse tree by applying grammar productions according to the input tokens. For the expression $id + id - id$ the parse tree determine the order in which operators are evaluated.

If the grammar does not specify operator associativity, the expression may be grouped incorrectly. For $id + id - id$ it may be interpreted as $(id + id) - id$.

5 Scenario : Operator precedence conflict.

Instead of the correct $(id + id) - id$ this result in incorrect evaluation because the operators are not associated properly.

Grammar rules directly influence the structure of the parse tree. A grammar that does not enforce associativity allow multiple valid parse trees for the same expression.

The grammar should enforce left associativity for addition and subtraction. For example

$$\begin{aligned} * E &\rightarrow E + T / E - T / T \\ * T &\rightarrow id \end{aligned}$$

This ensure expression are evaluated from left to right, producing the correct parse tree and accurate result.

2) Scenario : Invalid Expression Handling

The lexical analyzer correctly identifies the tokens in the expression $a * b$ as identifiers and operators. The syntax analyzer checks whether these tokens follow the grammar rules.

The error occurs because the grammar expects an operand after the '+' operator, but another operator '*' appears instead. Therefore, the token sequence violates the grammar making it impossible to construct a valid parse tree.

In predictive parsing, the parser detect the error when no valid production exist in the parsing table for the current input symbol. In shift-reduce parsing (LR), the parser detect the error when no valid shift or reduce action is available. Without proper recovery, the parser stop after reporting the error.

5 Scenario : operator precedence conflict.
The grammar

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

is ambiguous because it does not define operator precedence (associativity)

For the input $id + id * id$, two different parse trees are possible

$$* (id + id) * id$$

$$* id + (id * id)$$

The correct evaluation requires multiplication right precedence then addition since the grammar not specify the rule, the parser cannot determine the correct parse tree

operator precedence and associativity are to ensure expression are evaluated correctly

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow id$$

- * Multiplication has higher precedence than addition
- * Addition and multiplication are left associative

The parser should implement error detection and recovery techniques, such as panic-mode recovery, phrase-level recovery, or synchronization using Follow-sets. These methods allow the parser to continue analyzing the remaining input while providing meaningful error messages.

4) Scenario : Ambiguity in conditional statements

The Grammar

$$S \rightarrow \text{If } E \text{ then } S$$

$$S \rightarrow \text{If } E \text{ then } S \text{ else } S$$

$$S \rightarrow a$$

is ambiguous because the parser cannot determine which if statement an else belongs to. This is known as the dangling else problem.

When constructing the parsing table, conflicts arise because both productions begin with some prefix If E then S.

The parser cannot decide whether to reduce using the first production or continue parsing the second production containing else.

This ambiguity affects deterministic parsing by introducing parsing conflicts and producing multiple possible parse trees.

$$S \rightarrow M \mid U$$

$$M \rightarrow \text{If } E \text{ then } M \text{ else } M \mid a$$

$$U \rightarrow \text{If } E \text{ then } S \mid \text{If } E \text{ then } M \text{ else } U.$$